

Homigo Care Web Application Blueprint

- [HOMIGO CARE — WEB APPLICATION](#)
 - [Complete Build Blueprint for an AI Coding Agent \(Manus AI\)](#)
 - [Repository, Infrastructure, CMS Architecture, Three Responsive Shells, Screen-by-Screen Specification](#)
 - [0. Purpose, Working Method & When to Ask Questions](#)
 - [1. Repository & Infrastructure Setup](#)
 - [1.1 GitHub Repository](#)
 - [1.2 Vercel \(Primary Host — via Vercel MCP\)](#)
 - [1.3 Firebase — Web SDK, Same Project](#)
 - [1.4 Supabase — Same Project, JS Client](#)
 - [1.5 Cloudinary — Same Account, Same Presets](#)
 - [1.6 Environment Variables](#)
 - [1.7 AI Image-Generation Request Pipeline — 01_ai_images.json](#)
 - [1.8 The CMS Content Model — Read Carefully, This Powers §7.1](#)
 - [2. Global Design System — Single Source of Truth](#)
 - [3. Technology Stack \(Web — final, authoritative\)](#)
 - [4. Icons & Visual Assets — Web-Specific Decision Rule](#)
 - [5. Initial Load Experience \(Web’s Equivalent of a “Splash”\)](#)
 - [6. Authentication & OTP Flow \(identical pattern to the native apps\)](#)
 - [7. Complete Screen-by-Screen Blueprint](#)
 - [7.1 Public Site — No Login Required \(9 screens, all CMS-driven where noted\)](#)
 - [7.2 Patient Web App — Logged-In Screens \(mirrors the native apps’ P-xx set exactly\)](#)
 - [7.3 Professional Web App — Logged-In Screens \(mirrors the native apps’ PR-xx set\)](#)
 - [8. The Three Responsive Shells — Implementation Detail](#)
 - [8.1 Detection Logic](#)
 - [8.2 Desktop Shell \(components/shells/DesktopShell.tsx\)](#)
 - [8.3 Mobile-Android-Web Shell \(components/shells/MobileAndroidShell.tsx\)](#)
 - [8.4 Mobile-iOS-Web Shell \(components/shells/MobileIOSShell.tsx\)](#)
 - [8.5 Consistency Rule Across All Three](#)
 - [9. App-Wide Navigation Map \(conceptual, per shell’s own nav pattern\)](#)
 - [10. Data Sync Architecture \(Web-side summary — identical backend to Android/iOS\)](#)
 - [11. Build & Deployment Pipeline](#)
 - [12. Final QA Checklist Before Calling the Build “Done”](#)

HOMIGO CARE — WEB APPLICATION

Complete Build Blueprint for an AI Coding Agent (Manus AI)

Repository, Infrastructure, CMS Architecture, Three Responsive Shells, Screen-by-Screen Specification

0. Purpose, Working Method & When to Ask Questions

This is the **single, self-contained build order** for the **official Homigo Care website**, built as **one codebase that behaves like a full application on Desktop, and adapts into an app-like experience on Android-mobile-browsers and iOS-mobile-browsers**. It is the third of three client front-ends (alongside the native Android and iOS apps documented separately), sharing the exact same backend.

MCP servers available to the agent: GitHub, Supabase, Firebase CLI, Vercel, Cloudinary — same set as the Android and iOS projects, with Vercel now being the **primary hosting target** for this repo (not just a read-only awareness as in the native apps).

Working method — build in phases: 1. **Phase 0** — repository + infrastructure wiring (§1), a deployed “Hello Homigo” page live on Vercel. 2. **Phase 1** — Global design system (§2) + the three responsive shells (§8) working with placeholder content. 3. **Phase 2** — Public marketing/CMS-driven pages (§7.1): Landing, About, FAQ, Support Center, Legal — these can go live before login even exists, and prove the CMS pipeline end-to-end. 4. **Phase 3** — Full Patient web-app flow (§7.2), screen by screen, committing after each. 5. **Phase 4** — Full Professional web-app flow (§7.3), same approach. 6. **Phase 5** — Polish: responsive QA across all three shells, accessibility, performance.

Assets and information — ask, and keep working in parallel. Whenever the agent needs something only the founder can supply (a real icon/illustration not coverable by code, final legal text, a support phone number, a decision on an ambiguous flow), it should **ask for it explicitly and continue building everything else that doesn’t depend on it** — never block the whole project waiting on one asset. Missing visual assets always route through the 01_ai_images.json pipeline (§1.7) so the founder can batch-generate them while the agent keeps coding.

1. Repository & Infrastructure Setup

1.1 GitHub Repository

1. Using the **GitHub MCP**, create a new repository named exactly HOMIGO-CARE-WEB (if a repository with this name already exists, confirm with the founder before deciding whether to reuse or reset it, the same way it was handled for HOMIGO-CARE-IOS).
2. Folder structure:

```
HOMIGO-CARE-WEB/
├── app/
│   ├── (public)/
│   │   ├── page.tsx
│   │   ├── about/
│   │   ├── faq/
│   │   ├── support/
│   │   ├── legal/[slug]/
│   │   └── download/
│   ├── (auth)/
│   │   ├── login/
│   │   ├── signup/
│   │   ├── role-select/
│   │   ├── otp/
│   │   └── forgot-password/
│   ├── (patient)/
│   ├── (professional)/
│   └── layout.tsx
├── components/
│   ├── shells/
│   └── ui/
├── lib/
│   ├── firebase/
│   ├── supabase/
│   ├── cloudinary/
│   └── admin-api/
├── styles/
│   └── theme.css / tailwind.config.ts
├── public/
│   ├── icons/
│   ├── assets_pending/
│   │   └── 01_ai_images.json
├── .github/workflows/
├── docs/
│   └── this-blueprint.pdf
└── README.md
```

Next.js App Router
No-login-required routes
Landing page
Terms, Privacy, Refund Policy – dynamic, CMS-driven
"Get the app" page linking to Play Store / App Store
All P-xx routes
All PR-xx routes
Root layout: shell-detection + theme provider
DesktopShell.tsx, MobileAndroidShell.tsx, MobileIOSShell.tsx
Shared buttons, cards, badges (shadcn/ui-based, Tailwind-themed)
Auth + Firestore + FCM (web) wrappers
Supabase JS client + typed queries
Signed upload helpers
Typed fetch wrappers for the Admin Panel's API (OTP, CMS)
Design tokens – single source of truth (\$2)
Lucide is imported via npm, not files here
(mirrors app repos) – see \$1.7
optional extra CI checks; Vercel handles deploys natively on push

1.2 Vercel (Primary Host — via Vercel MCP)

1. Create a new Vercel project linked to HOMIGO-CARE-WEB, framework preset **Next.js**. Every push to main auto-deploys to production; every pull request gets its own preview URL — use preview URLs for the founder to check each phase before merging.
2. Add a custom domain once the founder has one (e.g. homigocare.pk for this public site; the separate Admin Panel project will later live at admin.homigocare.pk).
3. Set all secrets from \$1.6 as **Vercel Environment Variables** (Production + Preview + Development scopes) rather than committing them.

1.3 Firebase — Web SDK, Same Project

1. Register a **Web app** inside the same Firebase project used by Android/iOS (homigo-care-prod/homigo-care-dev). Copy the Firebase Web config object into environment variables (NEXT_PUBLIC_FIREBASE_*).
2. Enable the same sign-in methods already active (Email/Password, Google, Anonymous/Guest).
3. Add **Firebase Cloud Messaging for Web** (requires a VAPID key, generated in Firebase Console) so browser push notifications work on both desktop Chrome/Edge and Android mobile browsers (iOS Safari's web-push support is limited/version-dependent — treat web push as “best effort” on iOS Safari and rely on in-app/email notifications there as the reliable fallback).
4. Same Firestore usage rule as the native apps: **only** real-time/live collections (presence, bookings_live, chat_threads, notifications_feed, booking_status_events) — everything structured lives in Supabase.

1.4 Supabase — Same Project, JS Client

1. Point to the **same** homigo-care Supabase project as Android and iOS — one shared database across all three front-ends and the Admin Panel.
2. Add @supabase/supabase-js. Bridge the Firebase ID token to a Supabase session using the same JWT-claim-mapping function already set up for the native apps — a user's account and data are identical no matter which of the three front-ends (or the two native apps) they log in from.
3. Row Level Security policies already exist at the database level — no web-specific policy work needed beyond correct auth wiring.

1.5 Cloudinary — Same Account, Same Presets

Reuse `homigo_public_assets` (unsigned, app-wide icons/illustrations) and `homigo_user_documents` (signed, CNIC/degree/prescription/receipt uploads) exactly as configured for the native apps. On web, use the **Cloudinary Upload Widget** or a direct signed fetch upload from a Next.js **Server Action**/API route (never expose the API secret to the browser bundle).

1.6 Environment Variables

```
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_FIREBASE_PROJECT_ID=...
NEXT_PUBLIC_FIREBASE_VAPID_KEY=...
NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=...
NEXT_PUBLIC_CLOUDINARY_UNSIGNED_PRESET=homigo_public_assets
ADMIN_API_BASE_URL=https://admin.homigocare.pk/api      # server-side only, no NEXT_PUBLIC_ prefix where a secret is involved
NEXT_PUBLIC_MAPS_PROVIDER=osm
```

1.7 AI Image-Generation Request Pipeline — `01_ai_images.json`

Same schema and hard limits as the Android and iOS repos (20–50 entries per file, full subject/background/style/destination specification), living at `public/assets_pending/01_ai_images.json` in **this** repo. Destination paths point into `public/illustrations/`, `public/badges/`, `public/icons-custom/` (plain static files Next.js serves directly — simpler than the native apps’ asset catalogs, since web just needs files under `public/`). Because the web version often doubles as the **marketing site**, this file should also include a few web-only assets the native apps don’t need: an Open Graph / social-share preview image, a favicon set, and a hero illustration for the Landing page — batch these into the same file (or a `02_ai_images.json` if the 50-item limit is reached), following the identical JSON schema already established.

1.8 The CMS Content Model — Read Carefully, This Powers §7.1

Every FAQ entry, every Support Center field (phone/email/hours/address), every Policy/Terms/Privacy/Refund page, and the About Us content are **rows in the Supabase `cms_content` table**, not text hardcoded into any `.tsx` file. This is the same table the Admin Panel Blueprint document (built in the next chat) will describe the editing UI for — this document’s job is to make sure the **website correctly reads and renders** that data:

- `GET ADMIN_API_BASE_URL/api/cms/faq` → returns an ordered list of `{ id, category, question, answer, order }` → rendered on `/faq` grouped by category (Patient FAQ / Professional FAQ), in an expandable accordion.
- `GET .../api/cms/support` → returns `{ phone, whatsapp, email, hours, address, emergency_numbers[] }` → rendered on `/support` and reused in the footer of every page.
- `GET .../api/cms/legal/:slug` (terms / privacy / refund-policy) → returns rich-text/HTML content + version + `updated_at` → rendered on `/legal/[slug]` with the version/date visibly shown.
- `GET .../api/cms/about` → returns the About Us sections (mission, story, team, images) → rendered on `/about`.
- `GET .../api/cms/announcements` → active banners for the logged-in area’s top banner slot.
- **Cache every one of these calls** (Next.js fetch with `revalidate`, e.g. every 5 minutes) so the public pages stay fast, but never so long that an admin’s edit feels like it “didn’t work” — 5 minutes is a good default, with a manual “Revalidate now” webhook the Admin Panel can call after a save (`POST /api/revalidate?path=/faq`) for instant updates when needed.
- **The agent must not hardcode a single support phone number, email address, or paragraph of legal text anywhere in this codebase.** If real CMS content isn’t available yet during development, render the page against an empty/loading state and ask the founder for the Admin Panel’s live endpoint rather than inventing placeholder legal text that could accidentally ship to production.

2. Global Design System — Single Source of Truth

`styles/tailwind.config.ts` — extend Tailwind’s theme, never use an arbitrary inline hex value in a component:

```
colors: {
  'hg-primary-teal': '#1B6B63',
  'hg-dark-teal': '#0D4A44',
  'hg-header-emerald': '#0B5545',    // premium header bars only
  'hg-orange': '#F5823A',
  'hg-light-gray': '#F5F5F5',
  'hg-dark-gray': '#333333',
  'hg-medium-gray': '#888888',
  'hg-error-red': '#E53935',
  'hg-success-green': '#43A047',
  'hg-gold-elite': '#D4AF37',
  'hg-silver-top-rated': '#C0C0C0',
},
borderRadius: { 'hg-card': '15px', 'hg-button': '25px' },
fontFamily: {
  sans: ['Plus Jakarta Sans', 'system-ui', 'sans-serif'],
  urdu: ['Noto Nastaliq Urdu', 'serif'],
}
```

Every component imports these tokens via Tailwind utility classes (bg-hg-primary-teal, rounded-hg-card, etc.) — never a raw style={{ color: '#1B6B63' }}.

Typography scale: H1 22px Bold · H2 18px SemiBold · Body 14px Regular · Small/Caption 12px — matches the native apps exactly so a user switching between the app and the website never perceives a different brand.

Light mode only for v1 — no dark: Tailwind variants implemented yet, matching the native apps’ explicit decision.

Spacing scale: 4, 8, 12, 16, 24, 32 px. Minimum tap target 44x44px on mobile shells, comfortable 36px-minimum click targets on desktop (mouse precision allows slightly smaller, but never below accessibility minimums).

3. Technology Stack (Web — final, authoritative)

Layer	Choice	Reason
Framework	Next.js 14+ (App Router) , TypeScript	Industry standard, deploys natively to Vercel, supports the public marketing pages and the logged-in app in one project
Styling	Tailwind CSS	Fast, token-driven, matches the design system in \$2 exactly
Components	shadcn/ui (Radix-based, Tailwind-themed) for form controls, dialogs, accordions (FAQ), tabs	Accessible by default, fully restyleable to brand tokens — avoids reinventing accessible primitives
State	Zustand (lightweight global state: current user, active shell, cart-like combined-booking state) + React Server Components for data fetching where possible	Minimal boilerplate, no backend lock-in
Auth	Firebase Auth (Web SDK)	Same identity provider as the native apps
Realtime	Firebase JS SDK (Firestore + Messaging)	Live tracking, booking status, push notifications
Structured data	Supabase JS client	Bookings, profiles, documents, payments, reviews, CMS content
Images	next/image with the Cloudinary loader configured	Automatic responsive sizing/optimization for all illustration/icon assets
Maps	Leaflet + OpenStreetMap (via react-leaflet) for Desktop and Android-web shells; Apple MapKit JS for the iOS-web shell (visually matches native iOS Maps, still free)	Matches MAPS_PROVIDER decisions from the native apps, keeps each shell feeling native to its context
Hosting	Vercel	Free Hobby tier, auto-deploy, serverless functions/Server Actions for anything needing a secret
Charts	Recharts (Vitals graphs in Health Passport, Admin-style analytics reused later)	Lightweight, React-native (to React, not to be confused with React Native), themeable to brand colors

4. Icons & Visual Assets — Web-Specific Decision Rule

- Common UI glyph** (search, bell, star, home, settings, calendar, phone, checkmark, location pin, camera, share, close, menu, edit, delete, filter, chevron)? → **Lucide Icons** (lucide-react, MIT license, free) — this is the same family already used as the cross-platform fallback in the main brand blueprint, chosen specifically because it is license-safe everywhere (unlike SF Symbols, which stays iOS-native-app-only) and visually close enough to both Material Symbols and SF Symbols that the web version feels consistent with whichever native app a user also has installed.
- Brand-specific badge/category illustration/onboarding art/empty-state illustration?** → PNG/SVG asset, requested via 01_ai_images.json (\$1.7).
- Simple geometric badge that CSS/SVG can draw directly** (a plain circle with a checkmark, a colored dot, a progress ring)? → Inline SVG component — don’t spend an AI image slot on it.
- Favicon / Open Graph share image / app logo on the site** → generated from the founder-provided master logo file, placed at public/favicon.ico, public/icon.png, public/opengraph-image.png — never redrawn or AI-generated.
- On the **iOS-web shell** specifically, do not attempt to load real SF Symbols glyphs (Apple’s license restricts them to native Apple-platform apps) — Lucide icons styled with iOS-like stroke weight/sizing give the same visual family without a licensing problem.

5. Initial Load Experience (Web’s Equivalent of a “Splash”)

A website cannot force-play a video before the user sees content the way a native app splash can — use this pattern instead: 1. A minimal, fast **root loading skeleton** (app/loading.tsx) showing the centered app logo on a brand-teal background while the first route’s data streams in — this should resolve in well under a second on a reasonable connection, unlike the native apps’ deliberate few-second brand video. 2. If the founder specifically wants the **same splash video** to play once per browser session on the Landing page (marketing impact), implement it as an optional **muted, autoplaying, skippable** <video> element gated behind a “Watch our story” style trigger or a one-time session flag (sessionStorage) — but do **not** force it in front of the logged-in app experience (Home/Dashboard) the way the native apps do, since repeat web visitors expect instant access. Confirm this choice with the founder before building it — it is optional here, unlike the native apps where it is mandatory. 3. The **app logo** itself must appear identically (one shared <AppLogo /> component, reading from the same master asset as the native apps’ launcher icons) in: the site header (all three shells), the Login/Sign Up cards, the footer, the About page, and the favicon/Open Graph image.

6. Authentication & OTP Flow (identical pattern to the native apps)

- 1. Sign up (Email/Password or “Continue with Google”) via Firebase Auth (Web SDK) → app calls POST /api/otp/send on the Admin Panel API → 6-digit code emailed via the Admin Panel’s EmailJS SMTP configuration, stored in Supabase otp_codes with a 5-minute expiry.
- 2. 6-box OTP entry UI → POST /api/otp/verify → on success, profile marked email_verified = true → proceed to Profile Setup.
- 3. Google sign-in and Guest/Anonymous browsing skip this step. Guest browsing on web is especially useful for the **public marketing pages** (Landing, FAQ, Support, About) which never require login at all — only booking-related actions require an authenticated session.
- 4. Phone-number SMS OTP is intentionally deferred, same as the native apps — the UI exists, backed by this email flow for now.
- 5. Password reset follows the identical /api/otp/send → 6-box entry → new password pattern.

7. Complete Screen-by-Screen Blueprint

The website has **more screens than the native apps** because it also carries the public marketing/CMS-driven site (§7.1) that a native app doesn’t need. Every screen below renders inside **whichever of the three shells is active** (§8) — the content and logic are identical across shells; only navigation chrome (sidebar vs. bottom nav vs. tab bar) changes.

7.1 Public Site — No Login Required (9 screens, all CMS-driven where noted)

Route	Screen	Content & Key Components
/	Landing Page	Hero section (headline + CTA “Book Now” / “Get the App”), service category showcase (same badge icons as the apps), “Why Homigo Care” trust section (verified professionals, 3-phase assessment, GPS safety — pulled from the same brand messaging as onboarding), testimonials (real, moderated reviews once they exist — empty/placeholder-free section until real ones exist), footer with Support/FAQ/Legal links
/about	About Us	Fully CMS-driven — mission/vision, founder note, company story, team section with photos (GET /api/cms/about)
/faq	FAQ	Fully CMS-driven — accordion list grouped by Patient FAQ / Professional FAQ (GET /api/cms/faq), search box to filter questions client-side
/support	Support Center	Fully CMS-driven — phone, WhatsApp, email, hours, address, emergency numbers grid (GET /api/cms/support), a simple contact form that submits to a Supabase support_tickets table (visible to Support

/legal/terms	Terms & Conditions	Admin in the Admin Panel) Fully CMS-driven rich text (GET /api/cms/legal/terms), version/date shown
/legal/privacy	Privacy Policy	Same pattern, slug=privacy
/legal/refund-policy	Refund Policy	Same pattern, slug=refund-policy
/download	Get the App	Play Store / App Store badges + QR codes linking to the Android/iOS listings, brief feature highlights reusing the onboarding illustrations
/pricing	Pricing Overview	Public-facing version of the service pricing table (Budget/Premium per category) — sourced from the same cms_content/service-category tables the Admin Panel manages, so a price change there reflects here automatically

7.2 Patient Web App — Logged-In Screens (mirrors the native apps’ P-xx set exactly)

Route	Screen	Content & Key Components (web-specific notes)
/login, /signup, /role-select, /otp, /forgot-password	Shared Auth (S-05–S-09)	Card-centered on Desktop; full-bleed on mobile shells; same fields/flow as the apps, “Continue with Google” official button
/onboarding/who-is-this-for (P-08)	Patient Type	3 cards: Myself / My Child / Elderly Family Member
/onboarding/personal-details (P-09)	Personal Details	Name, CNIC + front/back upload (Cloudinary), DOB, Gender
/onboarding/address (P-10)	Address & Emergency Contact	Leaflet/MapKit-JS map picker, “Use my location” (navigator.geolocation), emergency contact fields
/home (P-11)	Home Dashboard	Desktop: multi-column layout — service grid left/center, upcoming booking + wallet summary sidebar right. Mobile shells: single-column, service grid, voice-search mic (Web Speech API where supported), red Emergency banner
/category/[id] (P-12)	Service Category	Sub-options with Budget/Premium price chips
/browse/[categoryId] (P-13)	Browse Professionals	Desktop: grid of cards with filters sidebar (rating, price, distance). Mobile: vertical list, filters in a bottom sheet/drawer
/professional/[id] (P-14)	Professional Profile	Bio, affiliation, badges, rating breakdown, moderated reviews, sticky “Book Now” (bottom-fixed on mobile, inline on desktop)
/booking/details (P-15)	Booking Details	Booking type tabs, date/time picker, notes field, “Add to combined booking”
/booking/health-passport-pin (P-16)	Health Passport PIN Share	PIN entry, “Share Emergency Info Only” toggle
/booking/supply-request (P-17)	Supply Pre-Request	Item list, per-item approve-cost checkboxes
/booking/payment (P-18)	Booking Summary & Payment	Price breakdown, live payment_accounts from Supabase, transaction ID + screenshot upload
/booking/confirmed/[id] (P-19)	Booking Confirmed	Success illustration, reference number
/booking/[id]/tracking (P-20)	Live Tracking	Real-time map (Firestore-streamed), ETA, call/message
/booking/[id]/in-progress (P-21)	Visit In Progress	Status timeline, SOS button (fixed position, always visible, exempt from scroll)
/booking/[id]/complete (P-22)	Visit Complete	Visit notes summary, “Rate this visit”
/booking/[id]/rate (P-23)	Rating & Review	5-category stars, text review,

/health-passport (P-24)	Health Passport	anonymous toggle 5 tabs (Basic Info/Medical History/Documents/Activity Log/Vitals with Recharts line graphs), PIN management Large SOS button (hold-to-confirm), one-tap emergency numbers — cache numbers client-side for a degraded-network fallback (true offline isn't possible on web the way it is on native, so clearly communicate "Call directly if this page won't load")
/emergency (P-25)	Emergency Screen	Test/panel picker, partner lab selector
/lab-test (P-26)	Lab Test Booking	Prescription upload/link, pharmacy selector
/medicine (P-27)	Medicine Order	Balance, transaction history, top-up with cashback tiers
/wallet (P-28)	Wallet	Code + native navigator.share() (falls back to copy-link), leaderboard, milestones
/referral (P-29)	Referral & Leaderboard	3 priority tiers, 30-day history
/notifications (P-30)	Notifications	Edit profile, addresses, payment methods, language toggle
/profile (P-31)	Profile & Settings	(Urdu/English, full RTL layout via dir="rtl"), About/Support links (CMS), Delete Account, Logout
/bookings (P-32)	Booking History	Filterable table (Desktop) / cards (mobile)

7.3 Professional Web App — Logged-In Screens (mirrors the native apps' PR-xx set)

Route	Screen	Content & Key Components
/pro/onboarding/field (PR-03)	Professional Field Selection	Category cards, then role list driven live from Supabase role_requirements
/pro/onboarding/documents (PR-04)	Document Upload	Dynamic form per selected role's required documents + CNIC, Cloudinary signed uploads with live thumbnails
/pro/onboarding/pending (PR-05)	Verification Pending	Status, review-time estimate, Support (CMS)
/pro/assessment/intro (PR-06)	Assessment Intro	3-phase explanation, single-device warning
/pro/assessment/test (PR-07)	Assessment — Live Test	Server-driven countdown, one question per view, browser back-navigation disabled/intercepted, copy-paste disabled on question text
/pro/assessment/results (PR-08)	Assessment Results	Score breakdown, badge-earned animation
/pro/onboarding/activate (PR-09)	Profile Activation	Availability hours, service radius, pricing tier, "Go Live" toggle
/pro/dashboard (PR-10)	Professional Dashboard	Desktop: multi-column (earnings + bookings + stats side by side). Mobile: stacked cards. Online/Offline toggle prominent
/pro/requests/[id] (PR-11)	Incoming Booking Request	Booking details, 10-minute countdown ring, Accept/Reject (push-notified via FCM web push where supported)
/pro/bookings/[id]/active (PR-12)	Active Booking	"Start Visit" (activates Safe Zone tracking), map navigation, call/message, shared Health Passport PIN
/pro/bookings/[id]/notes (PR-13)	Visit Notes (mandatory)	Vitals, care checklist + free text, supply items used, photo attachment, "Complete Visit"
/pro/earnings (PR-14)	Earnings & Payouts	Balance, payout request (Daily/Weekly/Monthly/On-demand, min PKR 500), history, payout account

8. The Three Responsive Shells — Implementation Detail

Every route in §7 renders its **business logic once** (shared hooks/data-fetching) but is wrapped in whichever shell matches the visitor's device. This is the core architectural requirement from the founder: one codebase, three distinct feelings.

8.1 Detection Logic

1. **Server-side first pass:** in `middleware.ts`, inspect the User-Agent header to set an initial guess (`android / ios / desktop`) as a cookie, so the very first server-rendered HTML already picks the right shell (avoids a flash of the wrong layout).
2. **Client-side confirmation:** on mount, re-check with `window.matchMedia('(max-width: 768px)')` for mobile-vs-desktop, and `navigator.userAgentData?.platform / navigator.userAgent` pattern-matching for Android-vs-iOS, correcting the cookie if the server guess was wrong (e.g. a desktop browser spoofing a mobile user-agent, or a tablet in an ambiguous size range). **Never trust the user-agent alone for anything security-relevant** — this detection only controls visual chrome, never permissions.
3. Store the resolved shell in a `ShellContext` (React Context) that every layout component reads from.

8.2 Desktop Shell (`components/shells/DesktopShell.tsx`)

- Persistent left sidebar navigation (Home, Health Passport, Bookings, Wallet, Profile — icons + labels, Lucide icons).
- Top bar: search, notification bell, profile avatar dropdown.
- Multi-column layouts wherever content allows (Home dashboard, Professional dashboard, Booking History table instead of stacked cards).
- Hover states on every interactive element (cards lift slightly on hover, buttons darken).
- Wide forms use two-column layouts (e.g. Personal Details: name + DOB side by side) rather than the single-column stack mobile needs.

8.3 Mobile-Android-Web Shell (`components/shells/MobileAndroidShell.tsx`)

- **Material 3** visual language: bottom navigation bar (5 items max: Home, Health Passport, Emergency (elevated FAB-style, red, center-positioned and slightly raised above the bar), Wallet, Profile), ripple effect on tap (active: Tailwind states simulating Material ripple), Roboto-influenced spacing (slightly denser than iOS).
- Single-column stacked layouts throughout, matching the native Android app's screen compositions from the Android Blueprint document almost 1:1, so a user switching from the Android app to the mobile website feels no disorientation.
- Uses `env(safe-area-inset-bottom)` padding under the bottom nav for gesture-nav Android devices.

8.4 Mobile-iOS-Web Shell (`components/shells/MobileIOSShell.tsx`)

- **Apple HIG** visual language: bottom tab bar with `backdrop-filter: blur(20px)` translucency (mimicking native iOS tab bars), SF Pro-style font stack (`-apple-system, BlinkMacSystemFont`), large-title-style page headers (big bold heading that shrinks on scroll, approximated with a `position: sticky + scroll-listener` collapsing effect).
- Rounded "continuous corner" card style (`border-radius: 15px` combined with a subtle super-ellipse approximation isn't natively possible in CSS, so use the standard 15px radius — visually close enough — do not over-engineer a true squircle in CSS).
- Uses `env(safe-area-inset-top)` and `env(safe-area-inset-bottom)` religiously (Dynamic Island / home-indicator safe zones), and `<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">` is mandatory in the root layout.

8.5 Consistency Rule Across All Three

Brand colors, the logo, typography scale, and every icon's *meaning* (not necessarily its exact rendering library) stay identical across all three shells — **only navigation chrome, spacing density, and native-feeling micro-interactions differ**. If a reviewer opens the same screen on Desktop, Android Chrome, and iPhone Safari side by side, they should immediately recognize it as "the same app," just dressed appropriately for each context.

9. App-Wide Navigation Map (conceptual, per shell's own nav pattern)

```

/home → /category/[id] → /browse/[categoryId] → /professional/[id] → /booking/details
→ .../health-passport-pin → .../supply-request → .../payment → .../confirmed/[id]
(near visit time) → .../tracking → .../in-progress → .../complete → .../rate
Persistent nav (shell-dependent chrome, same destinations): Home · Health Passport · Emergency · Wallet · Profile
Secondary: /lab-test, /medicine (from category grid), /referral (Wallet/Profile), /notifications (bell), /bookings (Profile)

/pro/dashboard → /pro/requests/[id] → /pro/bookings/[id]/active → .../notes (mandatory) → /pro/dashboard
Persistent nav: Dashboard · Earnings · Profile

```

10. Data Sync Architecture (Web-side summary — identical backend to Android/iOS)

Data Type	Source	Web Reads Via	Web Writes Via
Login session	Firebase Auth (Web SDK)	onAuthStateChanged	Firebase Auth SDK calls
Live GPS / booking status / presence / chat	Firebase Firestore	onSnapshot listeners	Firestore setDoc/updateDoc
Browser push notifications	Firebase Cloud Messaging (Web push, VAPID)	Service worker (firebase-messaging-sw.js)	(Admin Panel triggers sends server-side)
Profiles, bookings, reviews, payments, documents metadata, role requirements, wallet	Supabase Postgres	supabase-js .from('table').select(), RLS-scoped	.insert()/update(), RLS-enforced
CNIC/degree/prescription/receipt files	Cloudinary (signed)	Signed URL from a Next.js Server Action	Upload via signed preset from a Server Action (secret never reaches the browser)
App icon/illustration assets	public/ static files (preferred) or Cloudinary homigo_public_assets	next/image	N/A — placed by founder per §1.7
CMS content (FAQ, Support, Legal, About, pricing)	Admin Panel Vercel API / Supabase cms_content	fetch() with revalidate, per §1.8	N/A (read-only from this repo; edited from the Admin Panel)
OTP send/verify	Admin Panel Vercel API	fetch() to ADMIN_API_BASE_URL	Triggers only, no direct write

Same shared backend as both native apps — a booking made on the website is immediately visible in the Android app, the iOS app, and the Admin Panel, and vice versa.

11. Build & Deployment Pipeline

1. Every push to main auto-builds and deploys on **Vercel** (no separate CI file strictly required, though basic GitHub Actions lint/type-check/test steps are welcome as a pre-merge gate on pull requests).
2. Environment separation: a preview Vercel environment (per pull request) points at the homigo-care-dev Firebase project + a dev-schema flag in Supabase if needed; production points at homigo-care-prod — this protects the **zero-fake-data production rule** while allowing free experimentation on every PR preview.
3. After the Admin Panel project (built in the next phase) goes live, wire its POST /api/revalidate webhook so a CMS edit (FAQ, Support info, Legal text) can force-refresh this site's cached pages instantly instead of waiting for the default revalidation window.
4. Lighthouse/performance check recommended before each production merge — the public marketing pages (/ , /about, /faq) are often a new patient's very first impression of Homigo Care and should load fast even on a modest 3G/4G connection common in the target market.

12. Final QA Checklist Before Calling the Build “Done”

☐ Every component uses only the Tailwind tokens from §2 — zero inline hex colors, zero arbitrary font sizes.

- ☐ Every icon is either a Lucide icon, an inline SVG shape, or a founder-approved AI asset placed via §1.7 — no emoji used as icons, no SF Symbols referenced on web.
- ☐ All three shells (Desktop/Android-web/iOS-web) render every screen in §7 without any element clipping, at minimum widths 360px (small Android), 390px (iPhone), and 1280px (small desktop).
- ☐ Shell detection correctly distinguishes Android-web from iOS-web from Desktop in real browser testing (Chrome on Android, Safari on iPhone, Chrome/Edge/Safari on desktop) — not just simulated user-agent strings.
- ☐ Not a single support phone number, email address, FAQ answer, or paragraph of legal text is hardcoded anywhere — every one of those strings traces back to a GET /api/cms/* call.
- ☐ SOS/Emergency screen is reachable within 2 clicks/taps from anywhere in the logged-in app on every shell.
- ☐ Health Passport enforces the PIN gate and logs every access.
- ☐ Payment screen lists only the currently active payment_accounts fetched live — never hardcoded.
- ☐ No table/collection contains seed/demo data — every list (including the public Landing page’s testimonials section) shows a genuine empty state against a fresh backend, never fabricated reviews.
- ☐ public/assets_pending/01_ai_images.json is valid JSON, 20-50 entries, every destination_path matches a real reference used in a component.
- ☐ OTP send/verify round-trips correctly against the real Admin Panel /api/otp/* endpoints.
- ☐ Confirmed the website reads/writes the **same** Firebase project, Supabase database, and Cloudinary account as both native apps — a booking made on any one of the three platforms is visible from the other two and from the Admin Panel.
- ☐ Public pages (/ , /about, /faq, /support, /legal/*) are reachable and fully functional **without logging in**.
- ☐ Site is live on Vercel with auto-deploy from main confirmed working end-to-end.

Homigo Care — Web Application Build Blueprint · Desktop + Android-Web + iOS-Web, One Codebase · For Manus AI (GitHub + Supabase + Firebase CLI + Vercel + Cloudinary MCP) · Consolidated 2026